

简程 AI — AI 驱动的求职匹配工具

段韬 · 作品集项目文档

2026 年 6 月

30 秒电梯演讲

求职者海投简历，却不知道哪个岗位真正匹配、简历该改哪里。简程 AI 用语义理解替代关键词匹配，上传简历后 3 分钟内，告诉你最值得投的 5 个岗位，并给出具体到「哪句话该改成什么」的改写建议。不再盲投。



简程 AI 产品首页截图

简程 AI 产品首页

一、产品概述

简程 AI 是一个 AI 驱动的求职匹配工具。核心功能：用户上传简历、输入求职偏好，系统对岗位进行语义匹配，返回 Top 5 并给出针对性简历改写建议。

不是「建议强化 AI 经验」，而是「在第二段第一句加上你用 API 做过的具体事情，因为 JD 里这个词出现了三次」。

二、为什么要做这个方向

2.1 求职者的痛点

我在求职社区中观察到几个被反复提及的共性问题：

- **岗位推荐不精准**：平台推送的岗位匹配度低，「推荐理由」是模板化套话，没有解释「为什么匹配、差在哪里」
- **简历修改缺乏指导**：求职者知道要针对 JD 改简历，但没有工具告诉他们具体改什么、改哪里
- **决策依赖直觉**：判断一个岗位值不值得投，完全靠个人经验和当时的状态

2.2 现有方案的局限

主流招聘平台的核心匹配逻辑是**关键词覆盖**，而非语义理解。「数据分析」和「数据驱动」在关键词层面不相关，但在实际能力上是同一件事。这一根本局限导致真正匹配的岗位可能被过滤掉，不匹配的反反而被推到前面。

2.3 从双边视角看这个问题

这个匹配问题同样让招聘方痛苦：HR 收到大量用关键词堆砌的简历，但真正合适的人被埋没了。如果匹配能理解语义，未来向企业端提供一个「这个候选人为什么值得面试」的结构化摘要，就能打通双边价值。当前从求职者侧先验证匹配有效性，双边的平台思维是长期方向。

三、用户画像与场景

典型用户小林，应届硕士。做过数据分析、写过 SQL、实习用过 Python。手里有 30 个 JD，每次看都觉得自己差不多符合、又不完全符合，纠结很久。

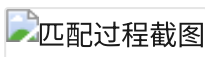
她上传简历，告诉系统想找「上海 AI 方向产品岗，20k+」，系统返回 5 个岗位——每个都有评分和具体理由：「你的 Gemini API 经验直接对应 JD 第二条，但你没有提到跨部门协作，JD 反复强调这个，建议补一段经历描述。」她照着改完，20 分钟就投递出去了。

四、产品方案

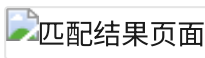
4.1 核心流程



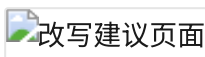
全流程不超过 3 分钟。



匹配过程：上传简历后实时语义解析与岗位匹配



匹配结果：三维度独立评分 + 具体推荐理由



改写建议：位置 + 内容 + 理由三元组结构

4.2 四个关键设计决策

① 偏好输入：自然语言，不是表单

用户在求职初期往往没有清晰的方向画像。一个自然语言输入框降低使用门槛，同时给系统足够匹配信号。

② 评分体系：三维度分开，不合并

技能匹配 / 经验匹配 / 方向匹配独立打分。用户可能技能高度匹配但方向不对口，综合分会掩盖这个差异。分开打分，让用户自己判断。

③ 改写建议：「位置+内容+理由」三元组

调试 Prompt 时最大的教训：早期模型输出方向性建议（「建议强调 AI 经验」），用户知道要改但不知道改哪里——**等于没有输出**。强制三元组后，建议从「方向」变成「可执行指令」。

④ 错误状态：用户永远看不到技术报错

每类技术错误都翻译成用户语言——「简历格式好像有点问题，换个 PDF 试试？」而不是「JSON parse error」。LLM 的不可靠性在产品层消化。

一个已知的产品哲学张力

三元组的「具体性」是核心竞争力，但越具体，对用户简历素材的要求就越高。对简历本身薄弱的求职者——他们恰恰最需要帮助——极度具体的建议可能从「赋能」滑向「提醒你缺什么」。这不是 MVP 要解决的，但值得标记。

五、AI 产品设计——把不可靠的 LLM 变成可靠的产品

5.1 Prompt 工程：三层约束

早期 Prompt 输出的是「方向性建议」，用户看完无法执行。三层约束解决：

层级	策略	说明
第一层	角色锚定	模型设定为「资深面试官」而非「职业规划顾问」——角色不同，输出完全不同
第二层	禁用词表	禁止「建议」「努力」「可以考虑」「加强」「提升」等模糊动词
第三层	格式约束 + Few-shot	强制输出「位置+修改内容+理由」三元结构；嵌入 2-3 个标准输出范例

关键经验：Few-shot 范例比纯文字约束有效得多。嵌入 2-3 个标准输出范例后，模型输出的可执行性显著提升——模型是通过模仿来学习的。

5.2 产品层兜底：三层防御



5.3 输出质量验证

对同一份简历+JD，连续调用 API 10 次，检验关键信息一致性。当前约 80% 的输出在核心匹配结论上保持一致，剩余 20% 差异主要体现在措辞层面。

测试中发现的典型 Badcase：在第 3 次和第 7 次测试中，模型将简历中的「用户研究」经验错误归类为「客服经验」。这提示 Prompt 需要加强对专业术语的界定——这是一个已知待优化项。

AI PM 最核心的命题就是：当 AI 出错时，用户应该看到什么？答案是——用户永远不应该知道 AI 出错了。

六、MVP 阶段的取舍

做了什么

- PDF 简历上传与 AI 解析
- 语义匹配 + 三维度评分
- 针对性改写建议（三元组结构）
- 浏览器本地历史记录
- 错误状态全覆盖

主动没做什么——以及为什么

没做	原因	计划
账户体系	MVP 验证的是匹配+改写有没有用	v2.0 引入轻量级账户（跨会话记忆需求出现时）
真实岗位爬取	先验证匹配算法有效性。MVP 用 30 条模拟岗位聚焦一个问题——AI 能否理解「数据分析」和「数据驱动」是同一件事？	v2.0 接入 RAG 获取实时岗位
简历模板 / 美化	聚焦内容匹配，不做格式优化	匹配功能稳定后考虑
社交 / 分享	核心链路跑通前，分支功能都是干扰	用户有自然分享需求时再评估

这些「不做」和「做了什么」同等重要——PM 的核心价值之一是判断什么不该加、什么时候该加。

七、项目产出



简程 AI 各页面截图展示

技术栈

层级	选型
前端框架	Next.js 16 + React 19 + TypeScript + Tailwind CSS
后端	Next.js API Routes
AI 服务	DeepSeek API
数据存储	Prisma ORM + PostgreSQL
部署	Vercel

八、衡量体系：上线后看什么

8.1 核心漏斗（会话内四步）

步骤	埋点事件	目标值
① 上传简历	resume_uploaded	—
② 查看匹配结果	match_results_viewed	≥70%（核心链路完成率）
③ 查看改写建议	rewrite_suggestion_viewed	≥65%
④ 点击投递	apply_clicked	>30%（投递转化率）

漏斗之前：访问→上传。若此步转化率极低，后续优化得再好也只影响一小撮人。MVP 暂不设目标，上线首周即应观测。

跨会话留存独立追踪：7 日回访率 >25%，14 日留存暂不设硬目标但持续观测。漏斗回答「这次体验顺畅吗」，留存回答「用户还愿意回来吗」。

8.2 「匹配成功」的三层定义

投递点击率高 ≠ 匹配成功——推荐「腾讯 PM」给任何人都被点，体现的是岗位吸引力而非匹配能力。
需三层交叉验证：

层级	观测指标	核心问题
行为层	核心链路完成率、投递转化率、改写建议采纳率	用户做完期望的动作了吗？
体验层	建议页停留时长、踩/赞（v1.1）	用户觉得有用吗？
结果层	7 日回访率、偏好调整行为	用户行为改变了吗？

三维皆  = 强匹配成功；行为  + 体验  + 留存  = 最危险的虚假繁荣。

8.3 数据看板总表

原则：只放「看到数字后会做不同决策」的指标。

指标组	指标	目标	频率	触发动作
漏斗	访问→上传	追踪中	日	<20% → 首页/上传有问题
漏斗	核心链路完成率	≥70%	日	连续 3 天 <50% → 全链路排查
漏斗	投递转化率	>30%	日	<15% → 匹配质量出问题
漏斗	匹配→查看改写	≥65%	日	<50% → 入口不显眼/价值不足
漏斗	改写→投递	≥55%	日	<35% → 匹配/岗位问题
匹配质量	改写建议采纳率	≥50%	周	<30% → 重构 Prompt
匹配质量	踩/赞（v1.1）	赞≥80%	周	<70% → 分组排查
匹配质量	Badcase 趋势	不增长	周	突增 → Prompt 漂移
增长留存	7 日回访率	>25%	周	<15% → 价值感不足
增长留存	14 日留存	追踪中	双周	落差 >50% → 关键流失窗口
技术健康	PDF 解析成功率	≥95%	日	<90% → 格式兼容问题
技术健康	API 响应时长	<15s	日	连续 >30s → 降级策略
技术健康	降级兜底触发比	<5%	日	>10% → 供应商问题

8.4 诊断维度：按建议类型拆分采纳率

三类建议对用户的能力要求完全不同：快速改进（改动词/表述）大概率可执行；强化点（补充经历细节）取决于是否真做过；重新定位（转方向）可能完全无法执行。

如果「快速改进」80% + 「强化点」40% + 「重新定位」5%，加权平均达标，但「重新定位」完全无效——这是区分「建议格式不好」和「建议不可执行」两种不同病因的唯一手段。建议 v1.1 起按类型拆分追踪，作为匹配质量的月度诊断项。

九、Prompt 迭代实验与阶段判断

9.1 A/B 实验：验证三元组 Prompt

维度	对照组	实验组
Prompt	方向性建议 Prompt（旧版）	三元组 Prompt（角色锚定+禁用词表+位置+内容+理由格式约束）
分流	用户级随机 50/50，同用户始终同组；≥2 周	
样本量	每组 ≥100 个完成查看改写建议的用户	
主指标	改写建议采纳率（复制/展开行为）	
护栏	整体投递转化率不降	

⚠ 前置条件

若「匹配→查看改写」转化率 <50%，应先优化入口而非跑 A/B——否则「无差异」可能是用户根本没看到，不是格式没用。

决策规则

结果	行动
采纳率↑ + 投递率不降	全量上线三元组
采纳率↑ + 投递率↓	检查是否「过度修改」导致简历失真
采纳率无差异	复盘格式约束 / Few-shot 范例
采纳率↓	检查理解成本，考虑简化

9.2 阶段跃迁：从 MVP 到 PMF 的数据化判断

跃迁	进入条件	需
MVP → v1.1（反馈机制）	核心链路完成率 ≥60% + 改写建议采纳率 ≥40% + 20 条反馈	≥200 会话
v1.1 → v2.0（接入 RAG）	Badcase 连续 4 周不增 + 回访用户 >50% 使用偏好调整	≥500 会话
v2.0 → v3.0（跨会话）	7 日留存 ≥25% + 14 日 ≥15% + 单用户 ≥3 次匹配占比 >10%	≥1000 会话

数据不能消除误判，但保证「做决策时知道自己赌的是什么」。

十、坦诚

执行层面：以上框架在 MVP 阶段无法运行——无账户体系 → 留存不准、模拟岗位 → 投递行为不可信、样本量不足 → A/B 条件不满足。

认知层面：产品建立在一个核心假设上——用户与 JD 的差距主要是**表达性遗漏**（做过但没写），而非结构性缺失（根本没做过）。三元组优雅地解决了前者，对后者无能为力。MVP 用模拟岗位验证此假设，但信号可能系统性偏乐观——模拟岗位是为用户简历挑选的，真实岗位不是。v2.0 接入真实岗位后，将优先通过建议类型间采纳率差异验证：用户面对的是「不会写」还是「没东西可写」。

面试官关心的不是「你有数据吗」，而是「你知道产品的核心假设是什么，并知道怎么用数据验证它吗」。本文档就是我的回答。数据可以等到上线后才有。**数据思维不能。**